Teoria dos Grafos

Bruno Pereira dos Santos

Aula baseada nos cursos de outros docentes da UFMG (Prof. Antonio Alfredo Ferreira Loureiro), USP (Prof. Paulo Feofiloff), UFBA, livros, artigos e outros.

13 - Single-Source Shortest Paths

Caminhos mínimos de única fonte

Slides baseados na aula do prof. Hamilton (UESC)

Motivação

- Objetivo: encontrar a rota mais curta entre dois pontos
  - Exemplo: de Jequié (BA) até Salvador (BA)
- Abordagem Ingênua (Ineficiente)
  - Enumerar todos os caminhos possíveis
  - Calcular a distância de cada rota
  - Escolher a menor
- Problema:
  - Número enorme de possibilidades
  - Muitas rotas irrelevantes (ex: desvios grandes)

Modelagem com grafos

- Modelo
  - Grafo direcionado e ponderado: $G=(V,E)$
  - Função peso: $w: E \to \mathbb{R}$
- Vértices → interseções
- Arestas → trechos de estrada
- Pesos → distâncias (ou custo, tempo, etc.)
  - Para um caminho $p=\langle v_0, v_1, \ldots, v_k \rangle$:

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

Modelagem com grafos

- Definição: Um caminho p de u até v é mínimo se: $w(p)=\delta(u,v)$, *onde* $\delta(u,v)$ é a distância do caminho mais curto entre u e v.

$$\delta(u,v) = \begin{cases} \min\{w(p) : u \rightsquigarrow v\}, & \text{se existe caminho} \\ \infty, & \text{caso contrário} \end{cases}$$

➜ Pode haver mais de um caminho mínimo

➜ Caso Especial: Grafos Não Ponderados

- Todas as arestas têm peso 1
- BFS encontra o caminho mais curto

Problema de caminhos mais curtos de origem única

- O foco é o problema de caminhos mais curtos a partir de uma única origem:
  - Dado um grafo $G=(V,E)$ e um vértice fonte $s \in V$, encontrar o menor caminho de s até todo vértice $v \in V$.

Variante	Objetivo	Observação
Destino único	Menor caminho de todo v até um destino t	Pode ser reduzido ao caso de origem única invertendo as arestas
Par único	Menor caminho entre dois vértices u e v	Resolve-se usando u como origem
Todos os pares	Menor caminho entre todo par (u,v)	Pode ser resolvido executando origem única para cada vértice, mas há métodos mais eficientes

Subestrutura ótima de caminhos mínimos

- Teorema: Seja (G, w) um grafo orientado e seja $P = (v_1, v_2, \ldots, v_k)$ um caminho mínimo de v_i a v_k. Então para quaisquer i, j com $1 \leq i \leq j \leq k$:
  - $P_{ij} = (v_i, v_{i+1}, \ldots, v_j)$ é um caminho mínimo de v_i a v_j.
- Demonstração
  - Suponha, por contradição, que existe um subcaminho P_{ij}' mais curto que o subcaminho P_{ij} de um caminho mínimo P.
  - Então, substituindo esse trecho, você obtém um caminho total menor que P.
  - Isso contradiz o fato de P ser um caminho mais curto.

Construindo Caminhos Mínimos

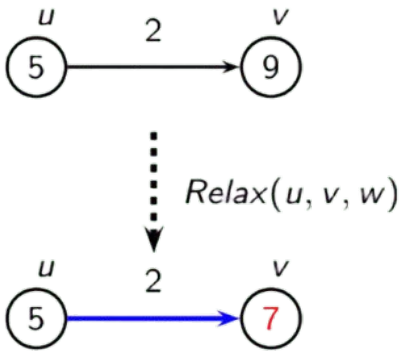
- A construção do algoritmo é baseada no algoritmo BFS (Busca em Largura).
- Cada vértice $v \in V(G)$ é associado a um predecessor $\pi[v]$.
- No final do algoritmo será construída uma Árvore de Caminhos Mínimos com raiz em s.
- Um caminho de s a v na Árvore é um caminho mínimo de s a v em (G, w)
- Para cada $v \in V(G)$ determinamos $dist(s, V)$, o peso de um caminho mínimo de s a v em (G, w) que representa a distância de s a v.
- Cada v E $V(G)$ terá associado $d[v]$ que é uma estimativa de $dist(s, v)$.

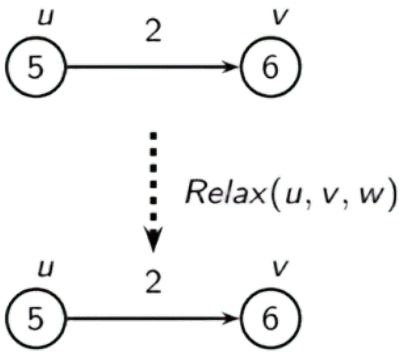
Iniciação de parâmetros

- $d[v]$ é uma estimativa superior para o peso de um caminho mínimos de s a v. Ou seja, $d[v] > dist(s, v)$.
- $d[v]$ tem a informação da menor distância que o algoritmo encontrou para um caminho de s a v.
- O caminho encontrado pode ser recuperado por meio dos predecessores marcados em $\pi[v]$.

```
Algoritmo IniciaOrigemUnica(G,s)
1. para v ∈ V[G]
2.   faça d[v] ← ∞
3.        π[v] ← NULO
4. d[s] ← 0
```

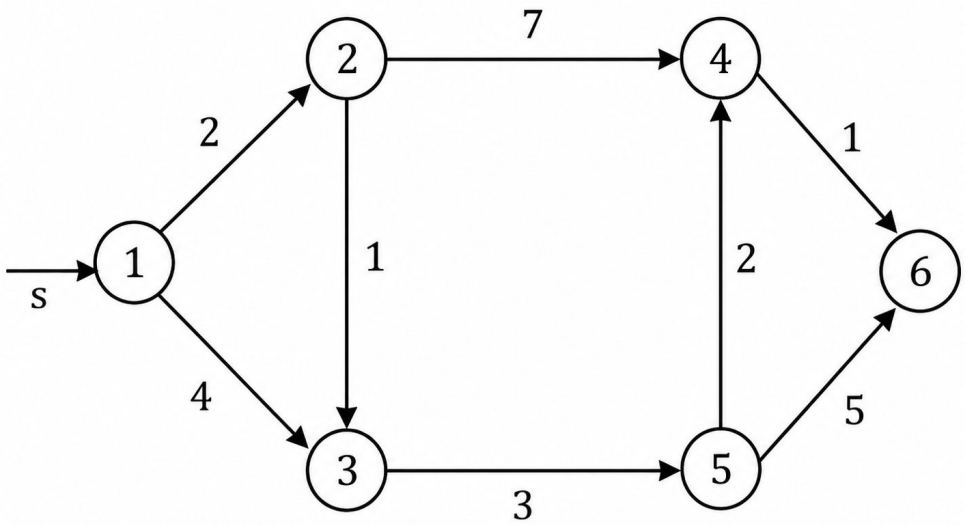
Iniciação de parâmetros





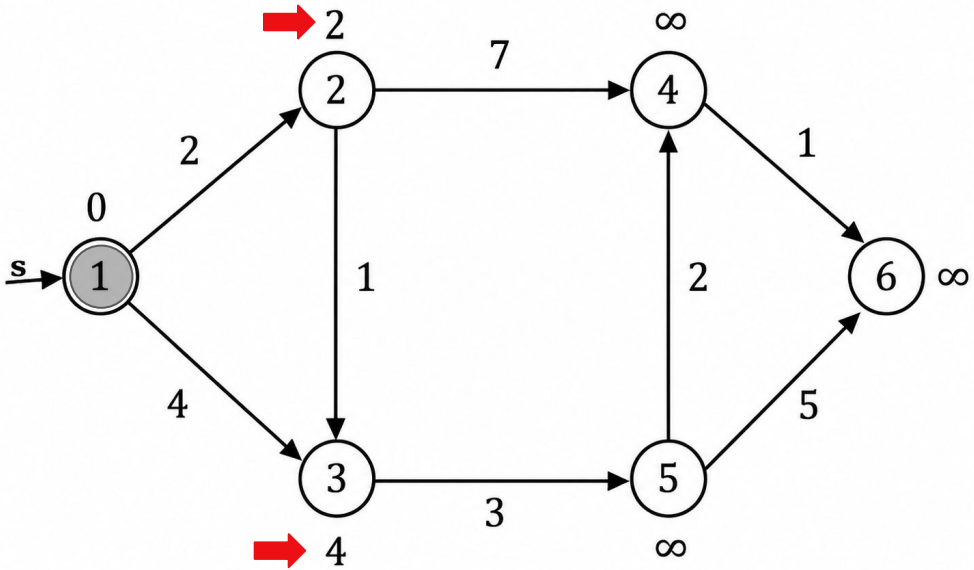

```
Algoritmo Relaxa(u, v, w)
1. se d[v] > d[u] + w(u, v)
2.   faça d[v] ← d[u] + w(u, v)
3.        π[v] ← u
```

Ideia dos algoritmos




```
Algoritmo Relaxa(u, v, w)
1. se d[v] > d[u] + w(u, v)
2.   faça d[v] ← d[u] + w(u, v)
3.        π[v] ← u
```

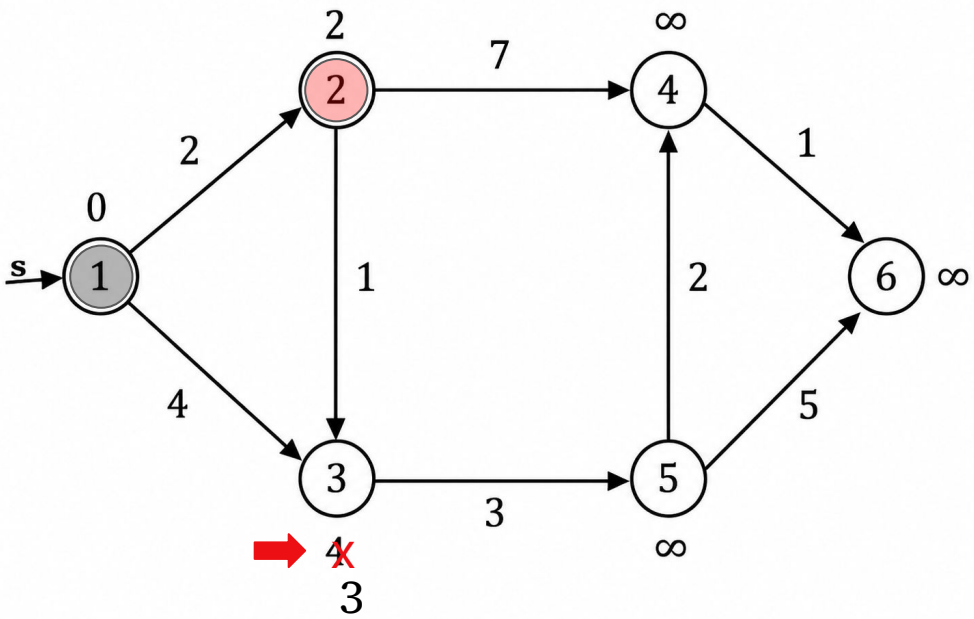
Ideia dos algoritmos (1)




```
Algoritmo IniciaOrigemUnica(G,s)
1. para v ∈ V[G]
2.   faça d[v] ← ∞
3.        π[v] ← NULO
4. d[s] ← 0
```

```
Algoritmo Relaxa(u, v, w)
1. se d[v] > d[u] + w(u, v)
2.   faça d[v] ← d[u] + w(u, v)
3.        π[v] ← u
```

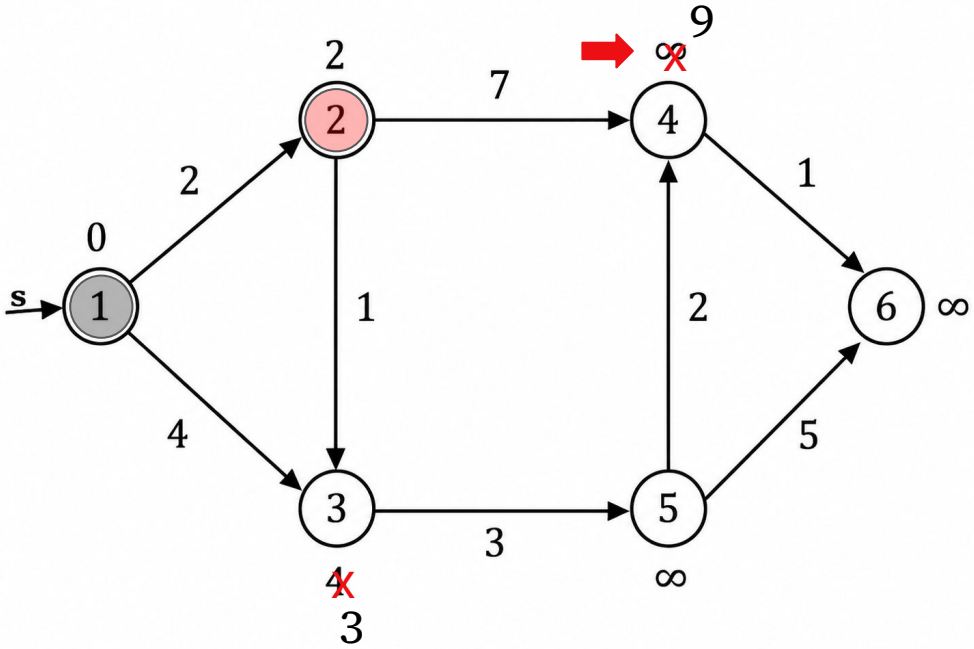
Ideia dos algoritmos (2)




```
Algoritmo IniciaOrigemUnica(G,s)
1. para v ∈ V[G]
2.   faça d[v] ← ∞
3.        π[v] ← NULO
4. d[s] ← 0
```

```
Algoritmo Relaxa(u, v, w)
1. se d[v] > d[u] + w(u, v)
2.   faça d[v] ← d[u] + w(u, v)
3.        π[v] ← u
```

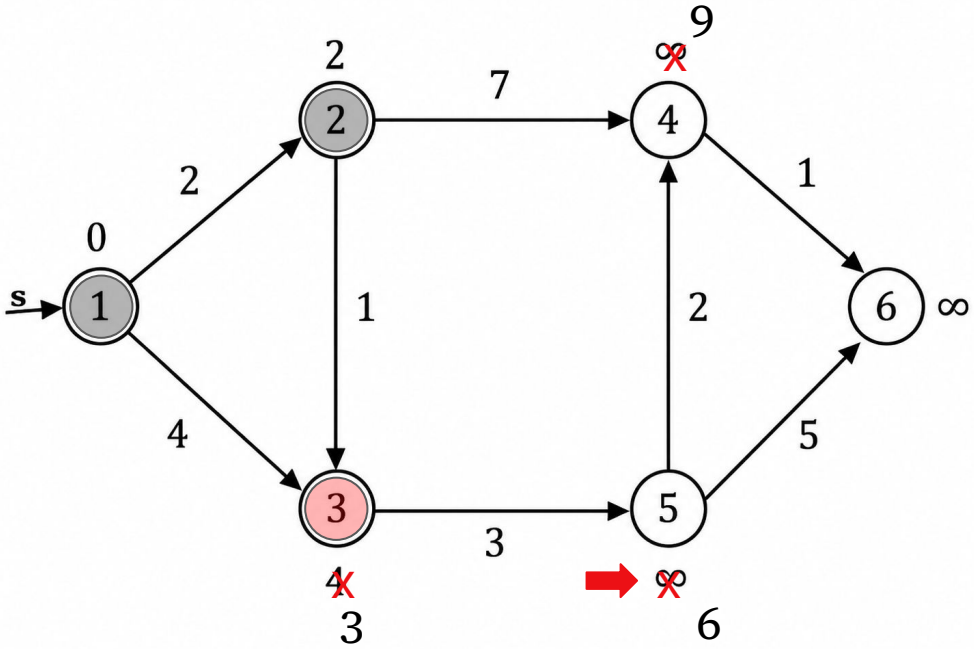
Ideia dos algoritmos (3)




```
Algoritmo IniciaOrigemUnica(G,s)
1. para v ∈ V[G]
2.   faça d[v] ← ∞
3.        π[v] ← NULO
4. d[s] ← 0
```

```
Algoritmo Relaxa(u, v, w)
1. se d[v] > d[u] + w(u, v)
2.   faça d[v] ← d[u] + w(u, v)
3.        π[v] ← u
```

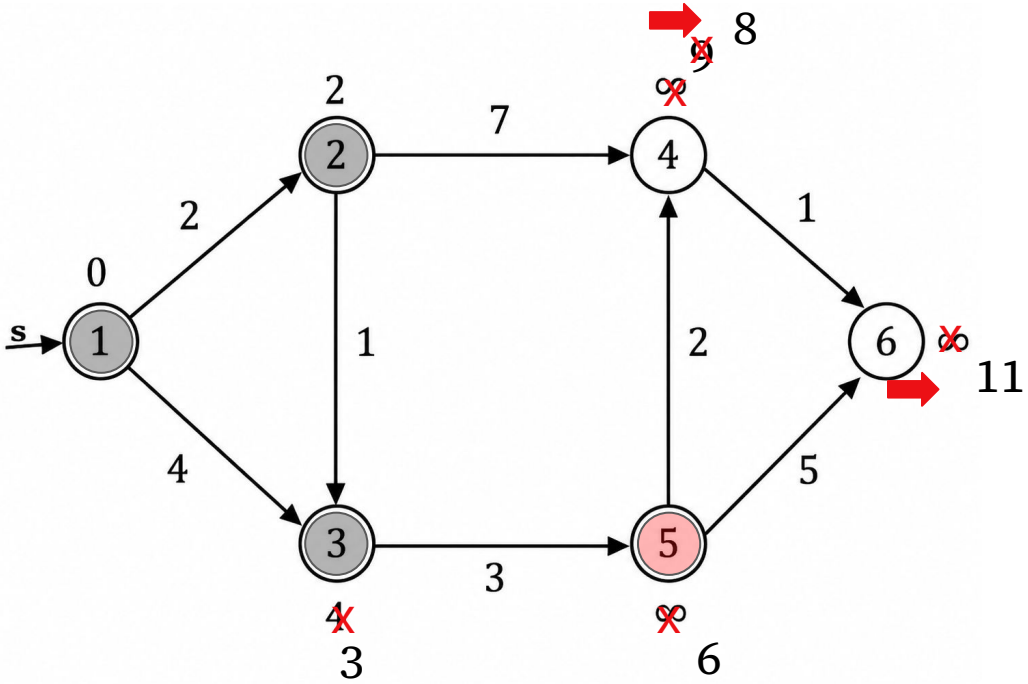
Ideia dos algoritmos (4)




```
Algoritmo IniciaOrigemUnica(G,s)
1. para v ∈ V[G]
2.   faça d[v] ← ∞
3.        π[v] ← NULO
4. d[s] ← 0
```

```
Algoritmo Relaxa(u, v, w)
1. se d[v] > d[u] + w(u, v)
2.   faça d[v] ← d[u] + w(u, v)
3.        π[v] ← u
```

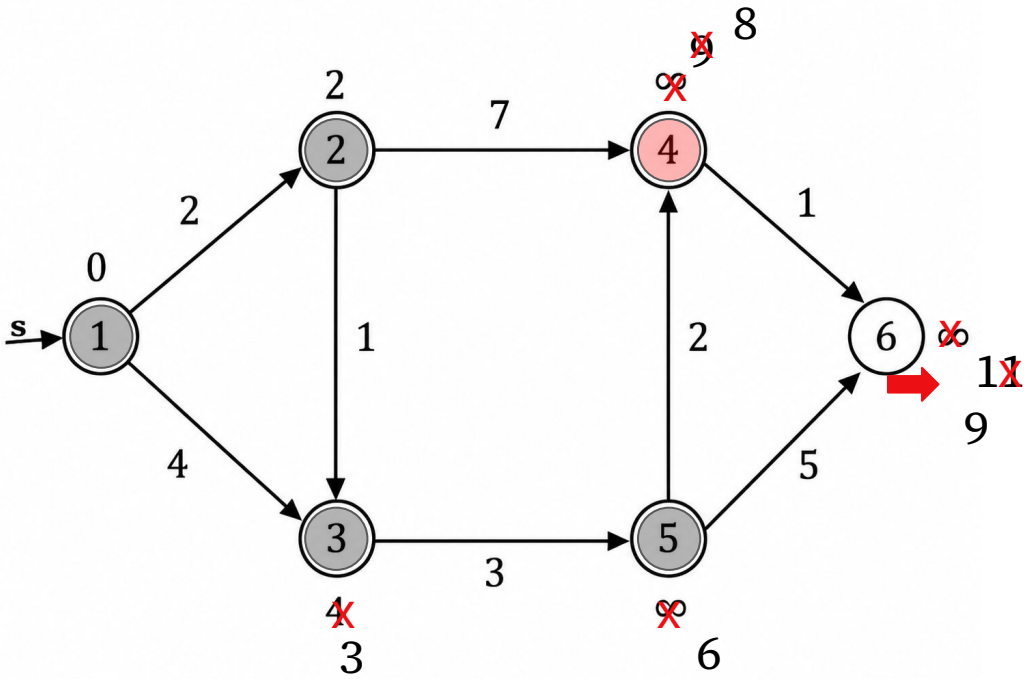
Ideia dos algoritmos (5)




```
Algoritmo IniciaOrigemUnica(G,s)
1. para v ∈ V[G]
2.   faça d[v] ← ∞
3.        π[v] ← NULO
4. d[s] ← 0
```

```
Algoritmo Relaxa(u, v, w)
1. se d[v] > d[u] + w(u, v)
2.   faça d[v] ← d[u] + w(u, v)
3.        π[v] ← u
```

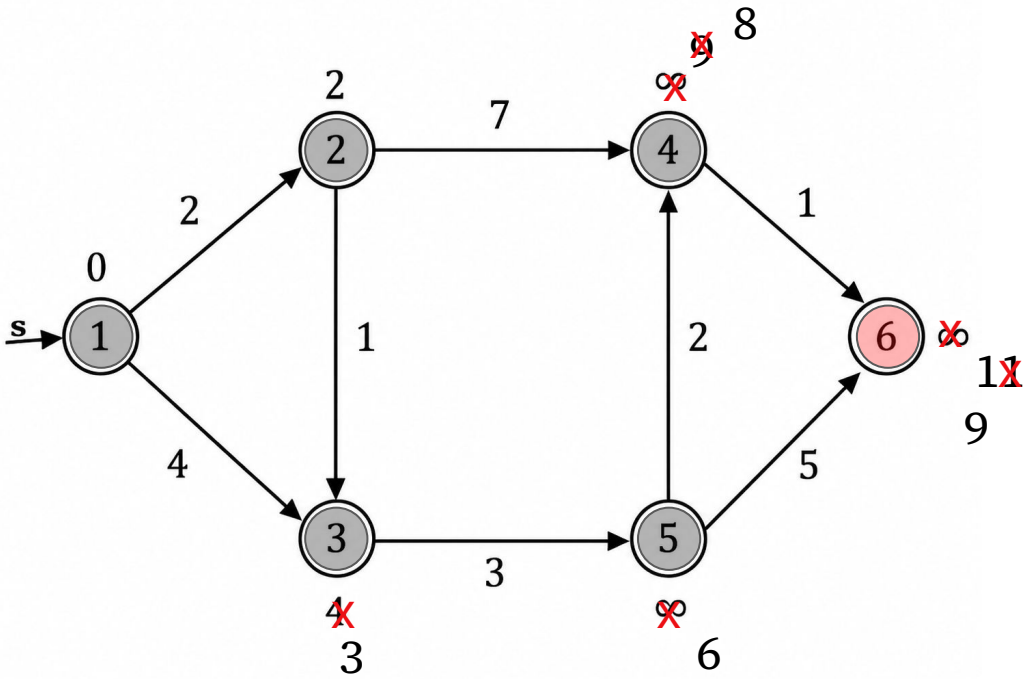
Ideia dos algoritmos (6)




```
Algoritmo IniciaOrigemUnica(G,s)
1. para v ∈ V[G]
2.   faça d[v] ← ∞
3.        π[v] ← NULO
4. d[s] ← 0
```

```
Algoritmo Relaxa(u, v, w)
1. se d[v] > d[u] + w(u, v)
2.   faça d[v] ← d[u] + w(u, v)
3.        π[v] ← u
```

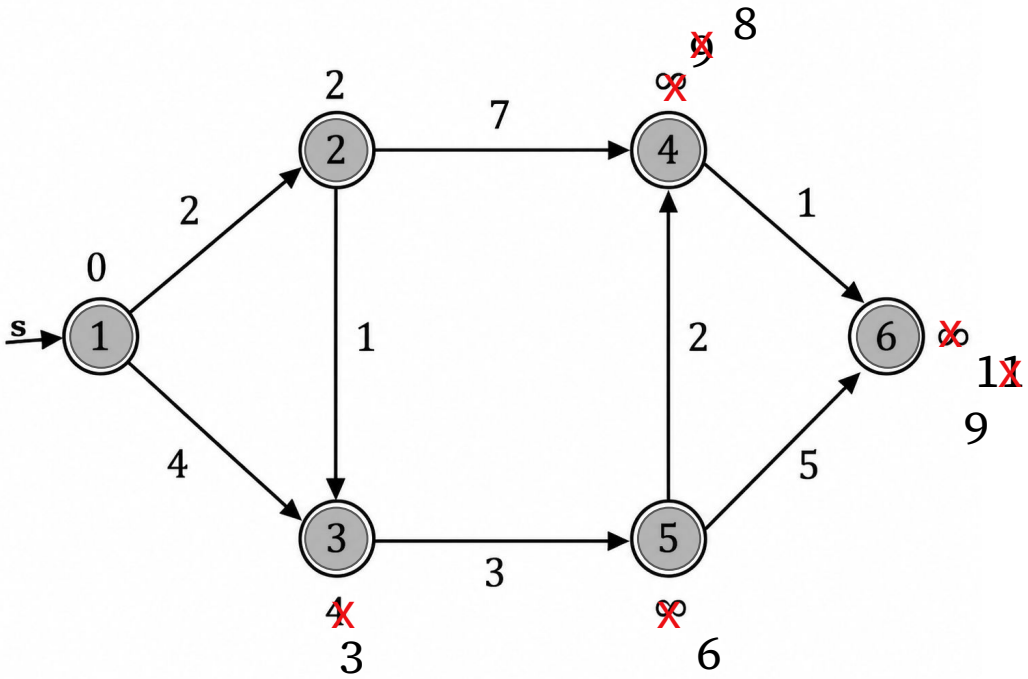
Ideia dos algoritmos (7)




```
Algoritmo IniciaOrigemUnica(G,s)
1. para v ∈ V[G]
2.   faça d[v] ← ∞
3.        π[v] ← NULO
4. d[s] ← 0
```

```
Algoritmo Relaxa(u, v, w)
1. se d[v] > d[u] + w(u, v)
2.   faça d[v] ← d[u] + w(u, v)
3.        π[v] ← u
```

Ideia dos algoritmos (8)




```
Algoritmo IniciaOrigemUnica(G,s)
1. para v ∈ V[G]
2.   faça d[v] ← ∞
3.        π[v] ← NULO
4. d[s] ← 0
```

```
Algoritmo Relaxa(u, v, w)
1. se d[v] > d[u] + w(u, v)
2.   faça d[v] ← d[u] + w(u, v)
3.        π[v] ← u
```

Algoritmos de Caminhos Mínimos de Unica Origem

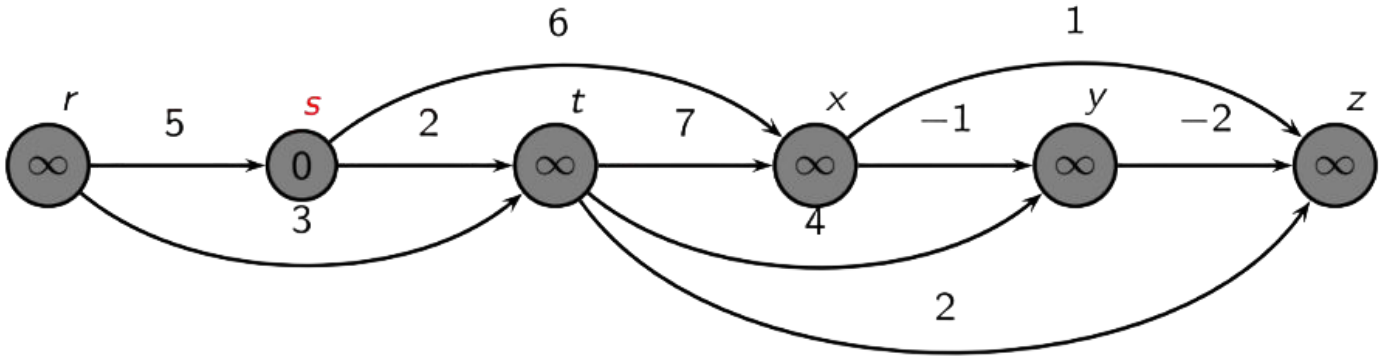
- Serão vistos três algoritmos que utiliza o relaxamento na busca de caminhos mínimos, cada algoritmo depende da natureza do grafo.
  - G é acíclico: **Algoritmo com Grafo Acíclico através de Ordenação Topológica.**
  - (G, w) não tem arcos de peso negativo: **Algoritmo de Dijkstra**
  - (G, w) tem arcos de peso negativo, mas não contém ciclos negativos: **Algoritmo de Bellman-Ford**

Caminhos mínimos em grafos acíclicos

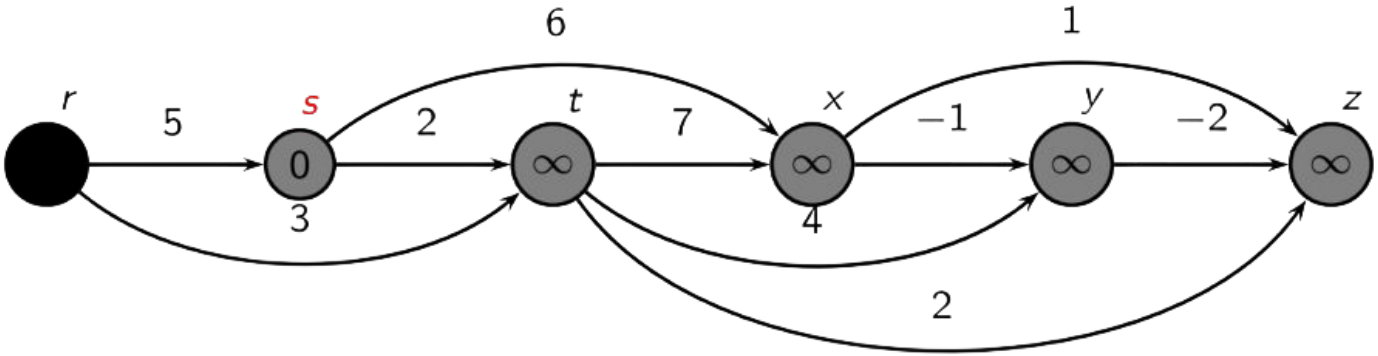
- **Entrada**: $G = (V, E)$ orientado e acíclico, com função peso w nos arcos, e uma origem s.
- **Saída**: Vetor $d[v] = dist\ (s, v)$ para $v \in V$ e uma Árvore de Caminhos Mínimos definida por $\pi[]$

```
Algoritmo CaminhosMinimosGAciclico(G, w, s)
1. Ordenacao Topologica(G)
2. IniciaOrigemUnica(G, s)
3. para u na ordem topológica
4.   faça para v ∈ Adj[u]
5.     faça Relaxa(u, v, w)
```

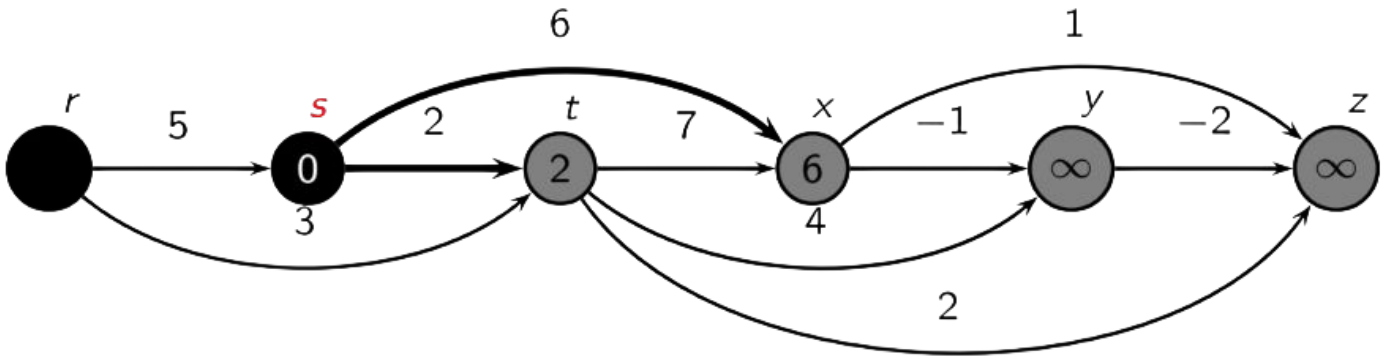
Exemplo



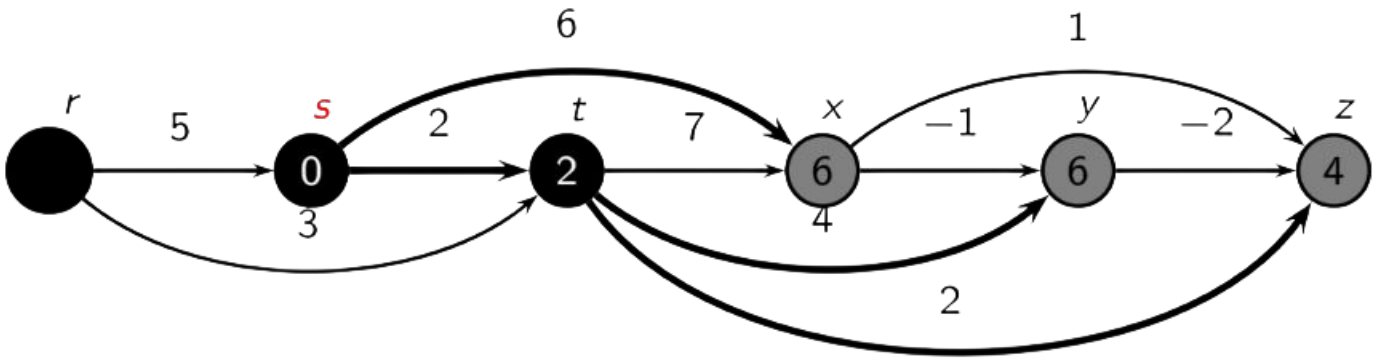
Exemplo (1)



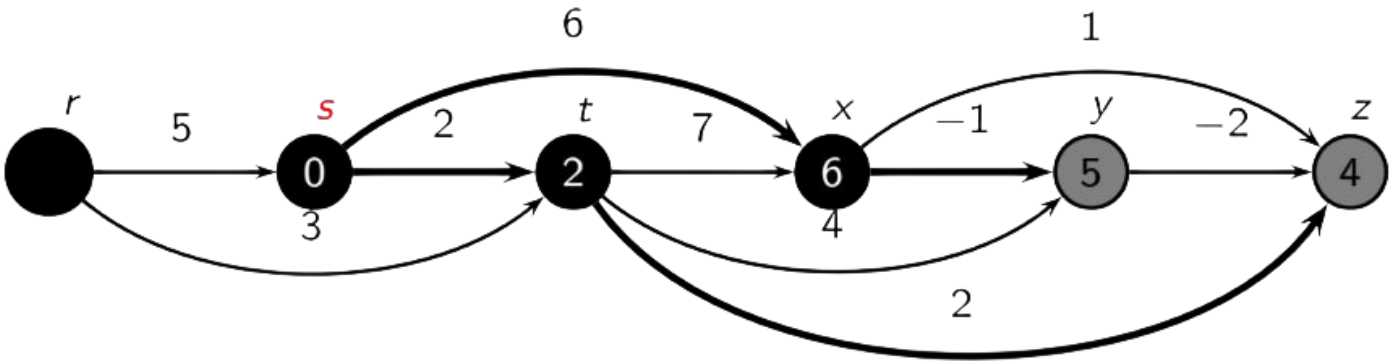
Exemplo (2)



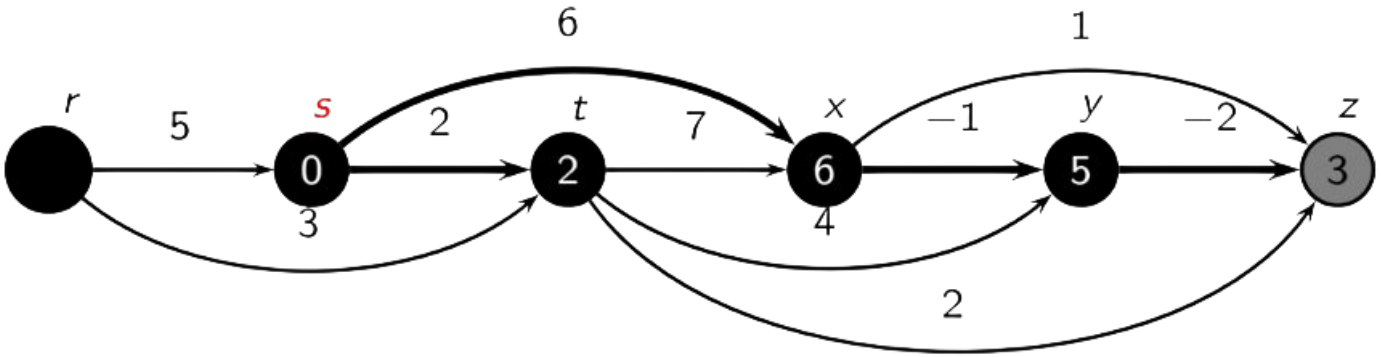
Exemplo (3)



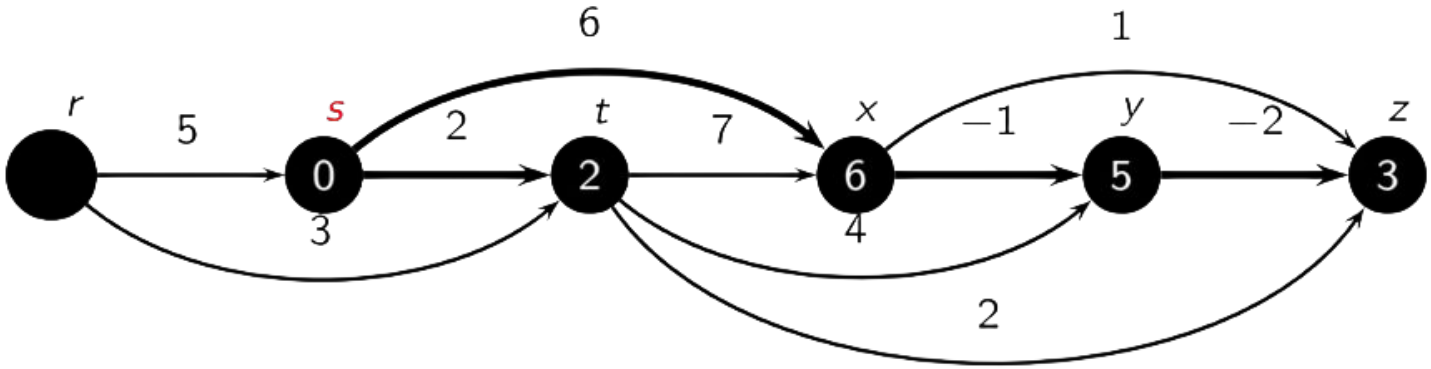
Exemplo (4)



Exemplo (5)



Exemplo (6)



Complexidade

- Complexidade
  - Linha 1 - O(V+E)
  - Linha 2 - O(V)
  - Linhas 3-5 - O(V+E)

```
Algoritmo CaminhosMinimosGAciclico(G, w, s)
1. Ordenacao Topologica(G)
2. IniciaOrigemUnica(G, s)
3. para u na ordem topológica
4.   faça para v ∈ Adj[u]
5.     faça Relaxa(u, v,w)
```

Algoritmo de Dijkstra

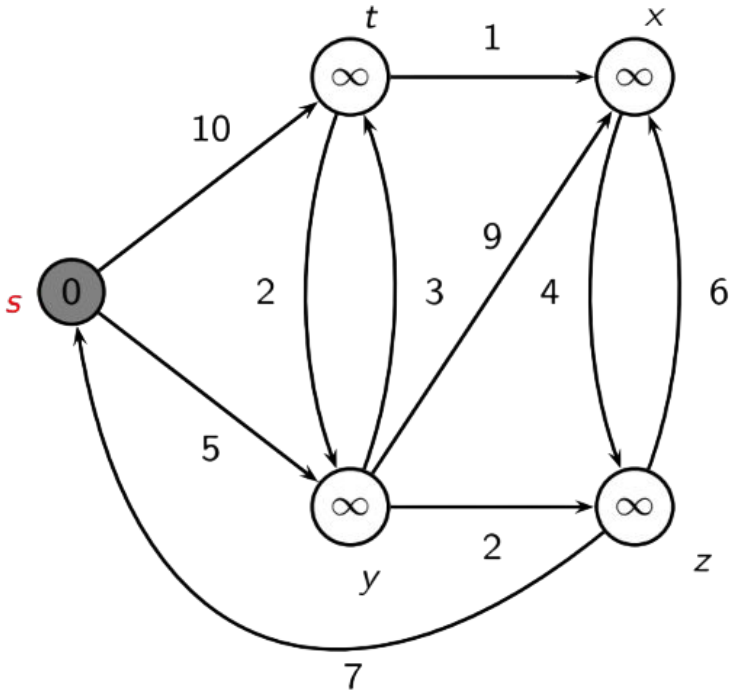
```
Algoritmo Dijkstra(G, w, s)
1. IniciaOrigemUnica(G, s)
2. S ← ∅
3. Q ← V[G]
4. Enquanto Q ≠ ∅
5.  faça u ← ExtrairMinimo(Q)
6.       S ← S ∪ {u}
7.       parav ∈ Adj[u]
8.           para Relaxa(u, v, w)
```

- Q é uma fila de prioridades
- S não é necessário, mas facilita na análise

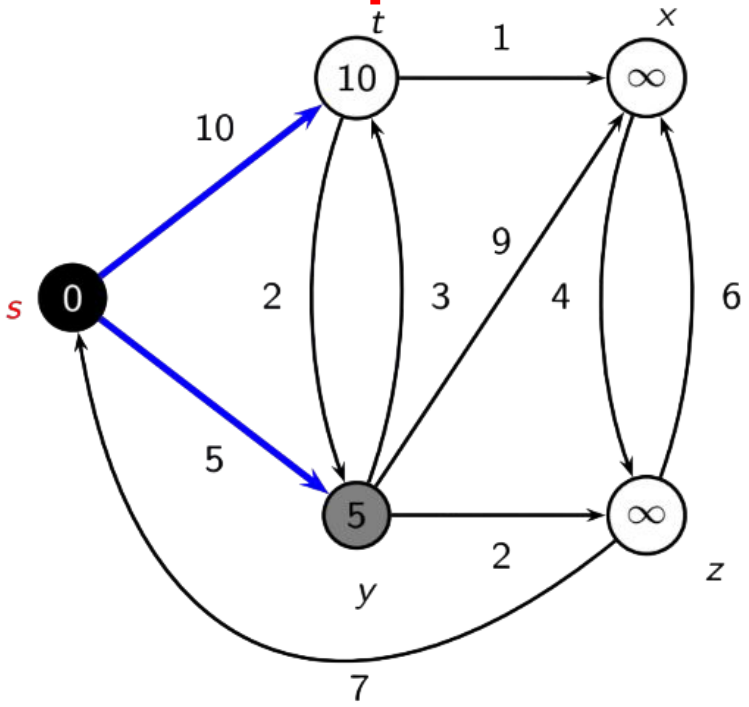
Algoritmo de Dijkstra

- Entendendo o algoritmo
  - Em cada iteração, o algoritmo Dijkstra
    - Escolhe um vértice u fora do conjunto S que esteja mais próximo a esse e acrescenta-o a S,
    - Atualiza as distâncias estimadas dos vizinhos de u e
    - Atualiza a Árvore dos Caminhos Mínimos.

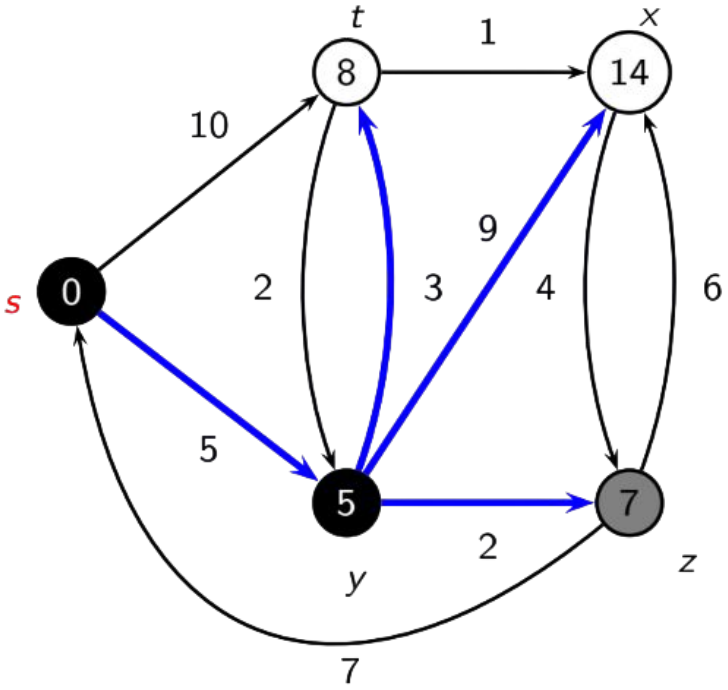
Exemplo



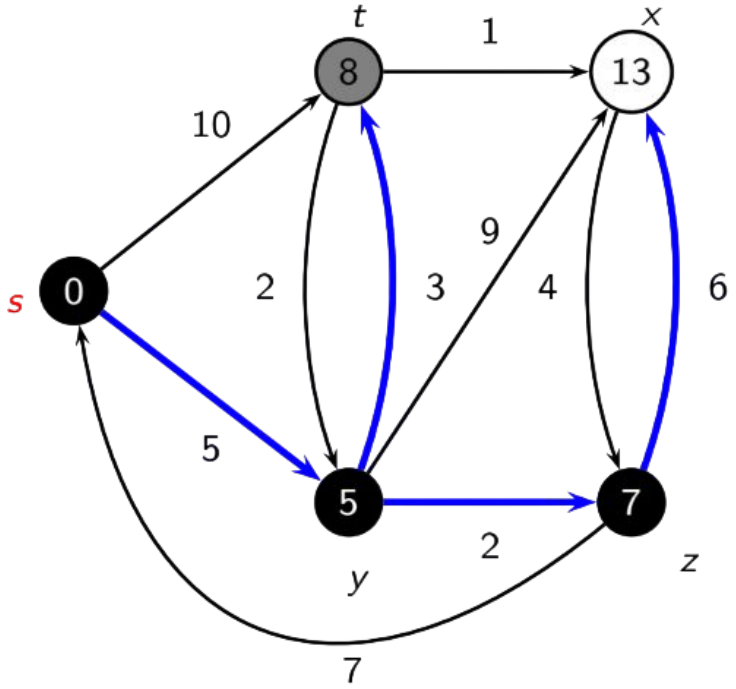
Exemplo (1)



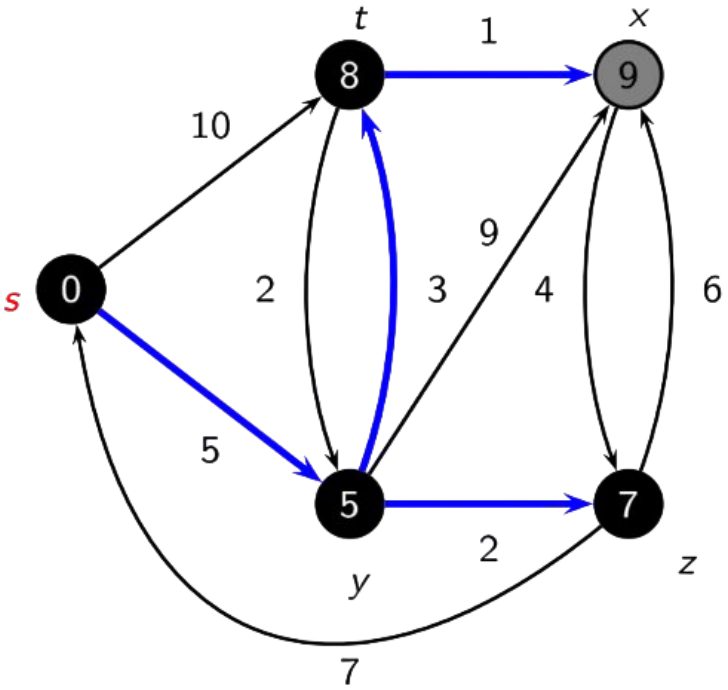
Exemplo (2)



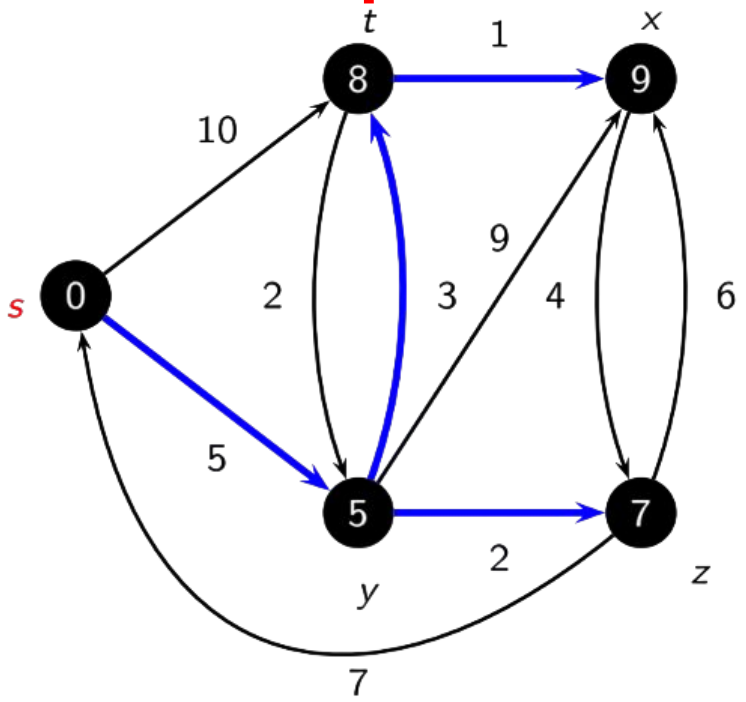
Exemplo (3)



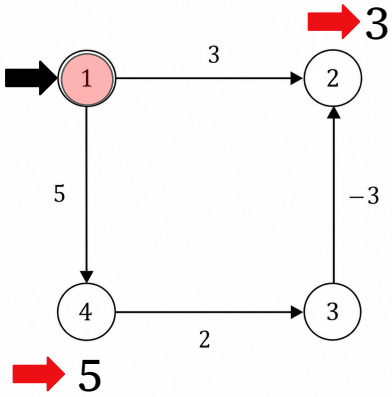
Exemplo (4)



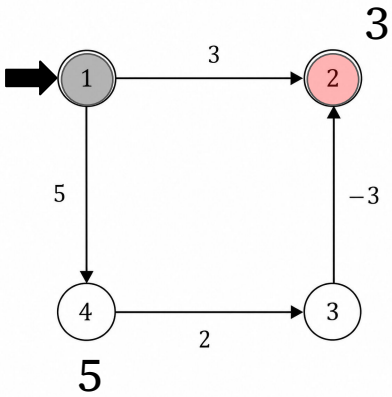
Exemplo (5)



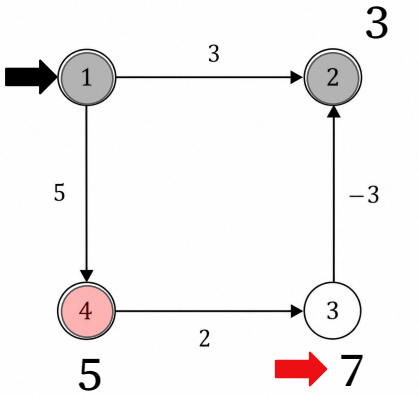
Exemplo [Dijkstra com peso negativo]



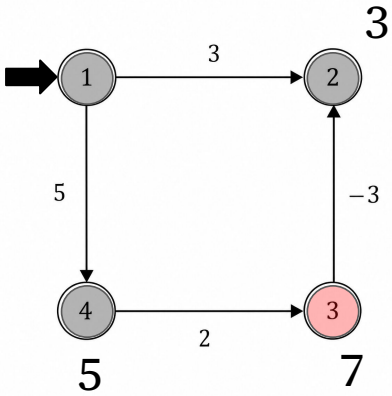
Exemplo [Dijkstra com peso negativo]



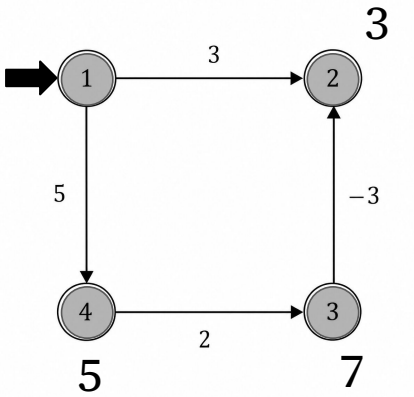
Exemplo [Dijkstra com peso negativo]



Exemplo [Dijkstra com peso negativo]

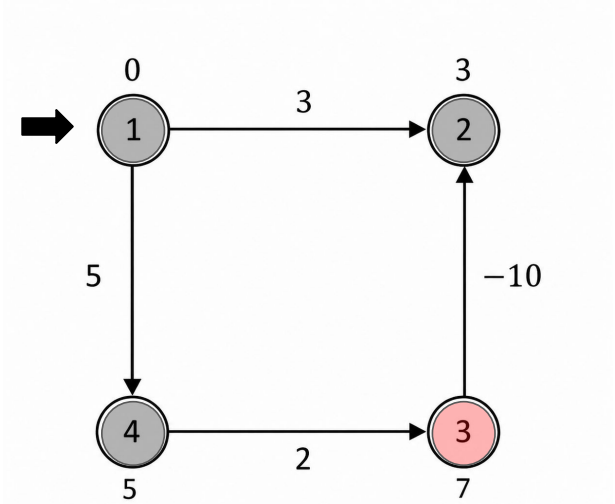


Exemplo [Dijkstra com peso negativo]

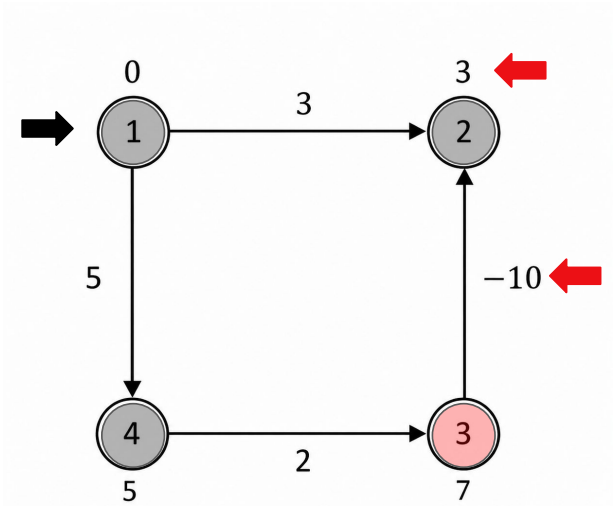




Exemplo [Dijkstra com peso negativo]



Exemplo [Dijkstra com peso negativo]





Complexidade

- Depende da implementação de Q. As operações em Q são:
  - **Inserir** (implícito na linha 3)
  - **ExtrairMinimo** (linha 5) e
  - **DiminuirChave** (implícito na linha 8, rotina Relaxa)

```
Algoritmo Dijkstra(G, w, s)
1. IniciaOrigemUnica(G, s)
2. S ← ∅
3. Q ← V[G]
4. Enquanto Q ≠ ∅
5.  faça     u ← ExtrairMinimo(Q)
6.       S ← S ∪ {u}
7.       parav ∈ Adj[u]
8.           para Relaxa(u, v, w)
```

Complexidade

- $|V|$ operações **Inserir**.
- $|V|$operações **ExtrairMinimo**.
- Cada vértice $u \in V(G)$ é inserido em S (no máximo) uma vez e cada aresta (u, v) com $v \in Adj[u]$ é examinada no Relaxa (u, v, W) (no máximo) uma vez nas linhas 7-8, são $|E|$ operações de **DiminuirChave**.

```
Algoritmo Dijkstra(G, w, s)
1. IniciaOrigemUnica(G, s)
2. S ← ∅
3. Q ← V[G]
4. Enquanto Q ≠ ∅
5.  faça     u ← ExtrairMinimo(Q)
6.       S ← S U {u}
7.       parav ∈ Adj[u]
8.           para Relaxa(u, v, w)
```

Complexidade

- No total são $|V|$ chamadas de ExtrairMinimo e $|E|$ chamadas de DiminuirChave.
  - Q como um **vetor**: $d[v]$ na posição v do vetor. Inserir e DiminuirChave são $O(1)$ e ExtrairMinimo gasta tempo $O(V)$, resultando em $O(V^2 + E) = O(V^2)$.
  - Q como fila de prioridade **Heap Mínimo**. Inserir, ExtrairMinimo e DiminuirChave gastam tempo $O(\log V)$, resultando em um total de $O((V + E) \log V)$.
  - Q como **Heap de Fibonacci (LINK)**. ExtrairMinimo é $O(\log V)$ e DiminuirChave é $O(1)$, resulta em uma complexidade menor: $O(V \log V + E)$.

Algoritmo de Bellman Ford

- Os algoritmos vistos consideram algumas restrições: Ou não são permitidos ciclos, ou não são permitidos arcos de peso negativo.
- O algoritmo de **Bellman Ford** permite relaxar estas restrições, são permitidos ciclos e arcos de peso negativo, no entanto não é permitido um ciclo de peso negativo (ou simplesmente ciclo negativo).

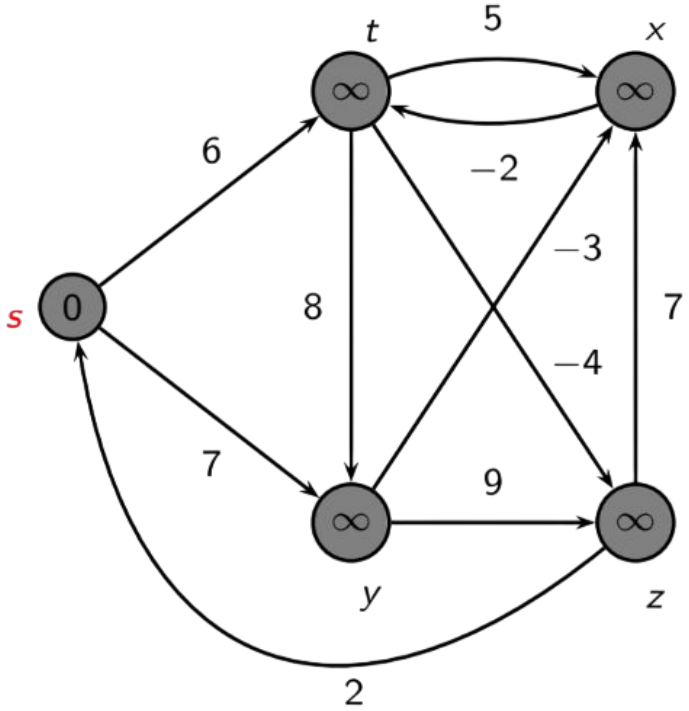
➜ Se houver um ciclo negativo o algoritmo irá parar indicando a presença de um ciclo negativo.

Algoritmo de Bellman Ford

- **Entrada**: Grafo (G, w) orientado e um vértice de origem s de G.
- **Saída**:
  - FALSO - se existe um ciclo negativo atingível a partir de s, ou
  - VERDADE - neste caso também devolve uma Árvore de Caminhos Mínimos com raiz s e o vetor $d[]$ com a distância de s a cada vértice.

```
Algoritmo Bellman-Ford(G,w,s)
1. IniciaOrigemUnica(G, s)
2. para i ← 1 até |V(G)| - 1
3.    faça para (u, v) ∈ E(G)
4.       faça Relaxa(u, v, w) para (u, v) ∈
E(G)
5.    faça se d[v] > d[u] + w(u, v)
6.       então retorne FALSO
7. retorne VERDADE
```

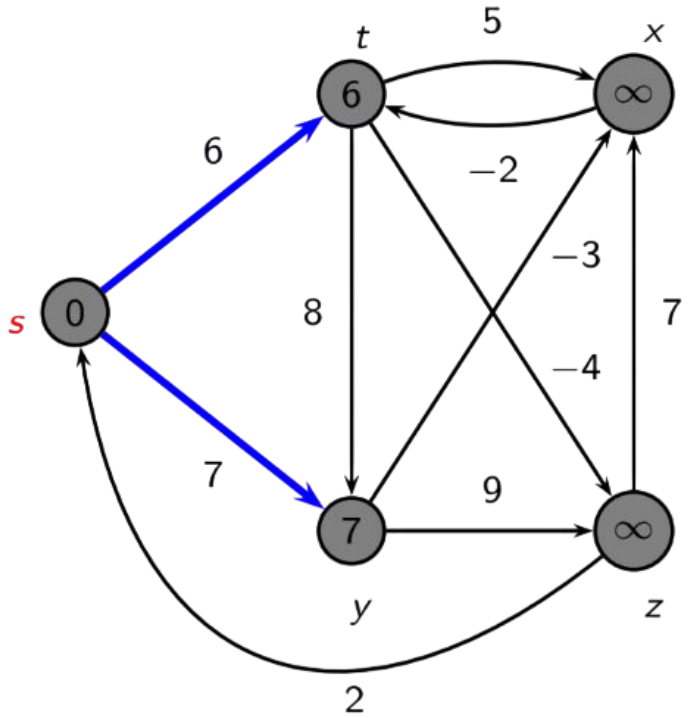
Exemplo



Ordem:

$(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y).$

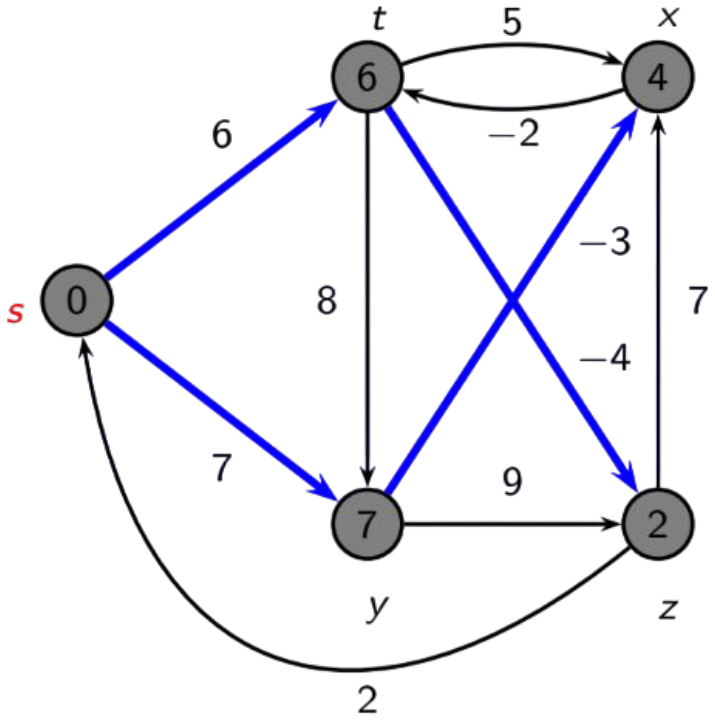
Exemplo



Ordem:

$(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y).$

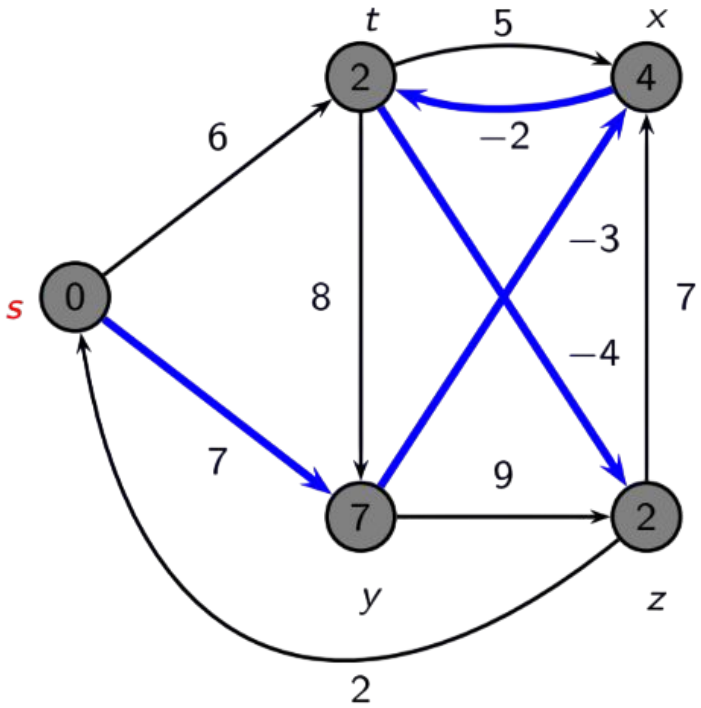
Exemplo



Ordem:

$(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y).$

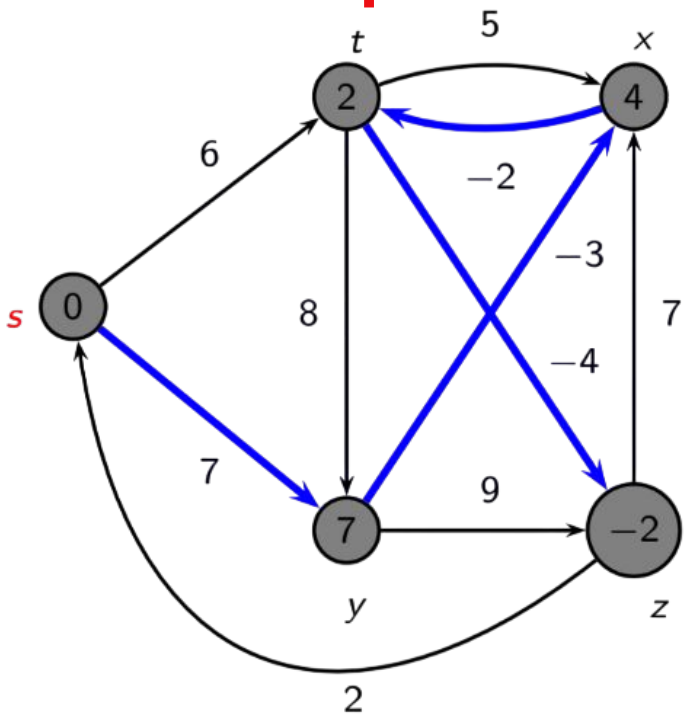
Exemplo



Ordem:

$(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y).$

Exemplo



Ordem:

$(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y).$

Complexidade

- Complexidade de tempo é $O(VE)$
- Com $|V|$-1 relaxamentos termina, se houver ciclo negativo, novo relaxamento deve mudar algum valor.
- Para corretude, precisamos mostrar que o algoritmo relaxa as arestas de um caminho na ordem correta.

Demonstração.

Um caminho que possui $|V|$ vértices, possui $|V| - 1$ arestas. Pela propriedade do Limite Superior se $d[v] = dist(s, v)$ seu valor nunca mais muda. Pela propriedade da Convergência, se $d[u] = dist(s, u)$ e (u, v) é um caminho mínimo de s a v, ao relaxar a aresta (u, v), $d[v] = dist(s, v)$. Por indução no número de vezes que as arestas são relaxadas, na i-ésima iteração, tendo $d[v_i] = dist(s, i)$ a aresta (v_i, v_{i+1}) ao ser relaxada, faz com que $d[v_{i+1}] = dist(s, v_{i+1})$. As arestas são relaxadas na ordem correta. □

Critério	Dijkstra	Bellman-Ford	DAG (Grafo Acíclico)
Tipo de grafo	Direcionado ou não	Direcionado ou não	Apenas DAG
Pesos negativos	Não permite* *(pode gerar resultados errados)	Permite	Permite
Detecta ciclos negativos	Não	Sim	Não (não há ciclos)
Complexidade	$O((V+E)log\ V))$ com heap Fib.	$O(VE)$	$O(V+E)$
Estratégia	Guloso (greedy)	Relaxamento repetido	Ordenação topológica + relaxamento
Número de relaxações	1 por aresta (idealmente)	Até (V-1) vezes por aresta	1 por aresta
Necessita estrutura extra	Fila de prioridade	Não necessariamente	Ordenação topológica
Eficiência	Muito eficiente	Mais lento	Mais eficiente possível
Aplicação típica	Grafos grandes sem pesos negativos	Grafos com pesos negativos	Dependências (ex: tarefas)

Teste os algoritmos

- LINK